Documentação e Explicação dos Comentários do Projecto

Este documento serve para explicar a estrutura principal e as secções de código que foram documentadas no projecto TeOdonto Angola.

1. Raiz da Aplicação (`App.tsx` e `index.tsx`)

A aplicação utiliza o **Expo** (React Native). O `index.tsx` é a porta de entrada que regista o componente principal (`App.tsx`).

`App.tsx`

O ficheiro `App.tsx` funciona como a "espinha dorsal" da aplicação. Todas as lógicas globais (Contextos) são inicializadas aqui. Foram comentadas as seguintes partes vitais da arquitetura:

- **Silenciamento de Avisos:** Código específico que ignora falsos positivos nas bibliotecas de compatibilidade web.
- **Configuração do React Native Toast Message:** A forma como desenhamos os balões de alerta de sucesso e erro na plataforma móvel e web.
- **Sincronização Offline:** Através da verificação do estado de rede (`useNetwork`), o `AppContent` tenta sincronizar a fila persistente quando não há internet.
- **Worker de Lembretes:** É aqui iniciado o `startReminderWorker`, responsável por recordar os pacientes (in-app, email ou WhatsApp) nas 24h anteriores à consulta.
- **Estrutura de Contextos (Providers):** A hierarquia obedece a uma ordem lógica — primeiro UI e Rede, depois Tema, Autenticação e dados de Utilizador (`DentistProvider`), seguindo-se finalmente a navegação (`AppNavigator`).

2. Configurações (`src/config/`)

`supabase.ts`

Ficheiro que define a conexão com o base de dados Supabase e a gestão do cliente, suportando inclusive um fall back para o cliente Admin (via role_key). Foi documentado que:

- **Cliente Anónimo vs Cliente Admin:** São mantidas duas instâncias independentes — a geral com Persistência da Sessão Activa que obedece ao *Row Level Security (RLS)*, e a Admin para uso irrestrito em rotas restritas.
- **Estratégia de Reconexão:** Configuração `realtime: { heartbeatIntervalMs: 15000 }` explicitamente comentada para explicar as mecânicas que gerem as escutas WebSocket quando a net cai.

`specialtyConfig.ts`

Documentação clara do dicionário `ESPECIALIDADES`, que formata toda a UI dependente da tipologia do dentista (como especialidades "Geral", "Ortodontia", "Endodontia", etc).
A documentação agora explica que o método `getEspecialidadeConfig` realiza "fuzzy matching" e que as constantes `color`, `colorLight` e `icon` orientam visualmente cada dashboard do dentista conforme a sua classe de atendimento.

3. Gestão Global de Estado / Contextos (`src/contexts/`)

`AuthContext.tsx`

O coração do front-end — foi detalhadamente comentado porque acumula múltiplas reponsabilidades complexas:

- **Sessões Únicas Concorrentes:** Lógica extensa para gerir os atributos `last_session_id`. Se a aba actual notar divergência com o Servidor (usando RealTime *Session Guard* ou *Interval Heartbeats*), irá disparar um kick para forçar o limite de um dispositivo activo por account.
- **MFA (Multi-factor Authentication):** A função `beginMfaChallengeIfRequired` foi explicada como interceptor: captura logins válidos, suspira o `setUser(null)` temporário e bloqueia-se aguardando o `verifyMfaCode`.
- **Inatividade de 5 minutos:** Um timeout universal (`INACTIVITY_TIMEOUT_MS`) que faz uso tanto do listener de janela (Web) quanto da API `AppState` no telemóvel para forçar o auto-logout e assegurar máxima robustez HIPAA.

`ThemeContext.tsx` e `NetworkContext.tsx`

Foram comentados com instruções concisas mas precisas.
No **ThemeContext**: foi explicado o fluxo da variável "automatic", onde o hook delegue a decisão para os settings do telemóvel.
No **NetworkContext**: clarificou-se que faz uso do `@react-native-community/netinfo` para espalhar reativamente pela árvore global React se há internet e que o estado obedece uma presunção online (optimistic true) inicial para prevenir flashes desnecessários da UI Offline.

Gerado automaticamente pela inteligência artificial. O processo de adição de comentários prossegue para os restantes ficheiros.